9주차 이 모델을 어떻게 만들었는지 어떻게 증명하는가

> **9주차 · 실험과 모델 관리**
>
> 이 장의 수치는 대부분 `practice/chapter9/` 의 코드를 실행해 얻은 값이다. 예시를 위해 지어낸 숫자는 없다. 인용한 값의 출처 파일은 출력 블록 첫 줄에 적었다.
>
> 한 가지 예외가 있다. MLflow가 직접 남긴 실험 기록과 모델 등록 결과는 이 강의 환경에서 다시 돌릴 수 없어 **과거 실행이 남긴 산출물에서 옮긴 값**이며, 그 블록의 출처는 `practice/chapter9/data/output/` 으로 적었다.

□ 7주차까지 만든 것은 데이터와 피처였고, 이 장에서 처음으로 **모델 훈련**이 등장함
  ○ 훈련이 등장하는 순간 따라오는 질문이 하나 있음 — 그 모델을 어느 데이터로, 어느 설정으로, 어느 코드로 만들었는가
  ○ 이 장은 그 답을 담당자의 기억이 아니라 시스템이 남긴 기록에 두는 방법을 다룸
□ 이 장도 설명과 실습을 나누지 않음
  ○ 절마다 실행할 명령이 나오고 그 아래에 실제 출력과 해석이 이어짐
  ○ 순서대로 실행하면 이 장의 수치를 직접 생성함

1. 오늘의 질문

□ 석 달 전에 배포한 모델을 지금 다시 만들 수 있는가. 무엇을 알아야 다시 만들 수 있는가.
□ 훈련 한 번에 무엇을 남겨야 "어떻게 만들었는가"에 답이 되는가.
□ 성능 기준을 통과한 새 모델을 운영에 넣지 않는 것이 옳은 경우가 있는가.
□ "지금 운영 중인 모델이 무엇인가"에 사람이 아니라 시스템이 답하게 하려면 무엇이 필요한가.

2. 이 장에서 실행할 것

□ 훈련을 여러 번 돌리는 일과 그중 하나를 운영에 넣는 일은 서로 다르고, 이 장은 둘을 나누어 다룸
  ○ 이 장에서 다루는 다섯 항목을 먼저 적음

이 장의 항목	무엇을 기록하거나 정하는가	다루는 절
run 하나 — 파라미터, 메트릭, 아티팩트(artifact)	훈련 한 번의 입력 설정·결과 수치·결과 파일	3
훈련 데이터의 지문(fingerprint) — sha256 해시	그 훈련에 쓴 데이터가 어느 것인지를 64자 값 하나로 고정	3·4
모델 레지스트리(registry)의 이름과 버전	배포 후보를 하나의 이름 아래 번호로 관리	5
champion 별칭(alias)	서빙 코드가 버전 번호 대신 조회하는 역할 이름	5
승격 게이트와 승격 검수 기록	어느 후보를 운영에 넣을지 정하는 규칙과 그 판정의 근거	5·6

표 9.1 이 장의 다섯 항목과 각각이 담당하는 것

□ 처음 보는 용어가 많으므로 읽는 순서를 정해 둠
  ○ 표의 용어를 지금 모두 외울 필요는 없음. 3~6절에서 용어마다 이 장의 실제 값(오차 1.0455, 모델 이름 `complaint_daily_forecaster` 등)을 보여 주고, 그때 이름을 붙임
  ○ 6절을 읽은 뒤 이 표로 돌아와 각 줄에 해당하는 값을 하나씩 말할 수 있으면 이 장을 이해한 것임
□ 표에서 셋째(레지스트리)와 넷째(별칭)는 다음 두 가지를 기억하면 됨
  ○ run은 수백 개 쌓일 수 있으나 그 전부가 배포 후보는 아님. 배포 후보만 따로 등록하는 저장소가 레지스트리임
  ○ 서빙 코드는 버전 번호 대신 `champion`이라는 별칭으로 모델을 불러옴. 별칭을 v1에서 v3으로 옮기면 서빙 코드를 고치지 않고도 운영 모델이 바뀜
□ 훈련에서 특히 확인할 것이 하나 있음
  ○ 코드·데이터·파라미터·환경이 같으면 훈련 결과가 같아야 함. 같은 설정으로 두 번 훈련했는데 오차가 서로 다르면, 어느 쪽이 그 설정의 결과인지 알 수 없음
  ○ 이 장에서는 두 번 모두 1.0455가 나오는 것을 4절에서 확인함
□ 이 장에서 실행할 코드는 넷이며 순서가 정해져 있음

순서	실행할 파일	하는 일	절	선행 조건
1	`9-2-training-pairs.py`	7주차 확정 집계에서 훈련 쌍과 지문 생성	3	없음
2	`9-3-train-and-rerun.py`	후보 둘 훈련·재실행 대조	4	없음
3	`9-4-gate-matrix.py`	게이트 규칙 셋으로 같은 후보 판정	5	없음
4	`9-5-review-record-check.py`	승격 검수 기록 점검	6	`data/output/ch9_promotion_record.json` (저장소에 포함)
선택	`9-1-mlflow-tracking.py`	같은 절차를 MLflow와 scikit-learn으로	3·5·6	`pip install -r code/requirements.txt`

□ 1~4번은 표준 라이브러리만 쓰므로 설치할 패키지가 없음(실습 검증 환경은 Python 3.13)

○ 선택 실습만 MLflow 3.14.0과 scikit-learn이 필요함

□ 앞 주차에서 쓴 적 없는 두 패키지를 먼저 소개함

○ **MLflow**: 머신러닝 실험 기록과 모델 관리를 맡는 오픈소스 파이썬 패키지임. 이 장에서 쓰는 기능은 둘임

– 실험 기록: 훈련 한 번(run)마다 파라미터·메트릭·아티팩트를 저장함(`log_params` , `log_metric` , `log_artifact` , 3절)

– 모델 레지스트리: 모델에 이름과 버전 번호를 붙이고 `champion` 같은 별칭을 관리함(5절)

○ **scikit-learn**: 선형회귀 같은 모델을 만들어 주는 파이썬 머신러닝 패키지임. 이 장에서는 `LinearRegression` 모델 하나를 훈련하는 데만 씀

○ MLflow는 모델을 훈련하는 도구가 아니라 훈련 결과를 기록하는 도구임. 5주차의 Spark가 계산을, 6주차의 Airflow가 실행 순서와 상태 기록을 맡았듯 역할이 다름

○ 1~4번 실습은 두 패키지 없이 같은 계산을 Python 표준 라이브러리로 직접 하고, 선택 실습이 그 값을 MLflow와 scikit-learn으로 다시 만들어 일치하는지 확인함(4절 출력 [4])

> - **선택 실습은 패키지를 설치한 뒤에 돌린다**
> - `9-1-mlflow-tracking.py` 는 mlflow와 scikit-learn이 있어야 돌아간다. 없으면 아무것도 만들지 못하고 끝난다.
> - 실행이 실패해도 다른 실습의 산출물은 남는다. 이 스크립트는 자기가 다시 만드는 실험 기록(`mlflow.db` · `mlruns/`)만 비우고, 산출물 JSON은 계산이 끝난 뒤 덮어쓴다.

3. run 하나에 무엇을 남기는가

□ 훈련을 한 번 돌릴 때마다 기록 한 묶음을 만들며, 이 묶음을 **run**이라 함
  ○ 이 장에서 훈련을 두 번 돌렸으므로(`baseline_mean` , `linear`) run이 둘 생김
  ○ **실험(experiment)** 은 관련된 run들을 모아 부르는 이름임

□ run 하나에는 세 종류를 나누어 적음
  ○ **파라미터(params)**: 훈련에 넣은 설정. 글자나 숫자로 적음. 예: `model_type = LinearRegression`
  ○ **메트릭(metrics)**: 훈련 결과로 나온 숫자. 이 장에서는 훈련 오차인 `train_mae` 하나임. MAE(평균 절대 오차)는 예측과 실제의 차이를 절댓값으로 바꿔 평균한 값이며, 작을수록 잘 맞음. 예: `linear` 는 1.0455
  ○ **아티팩트(artifacts)**: 결과로 남기는 파일. 예: 훈련에 쓴 CSV 파일, 저장한 모델 파일

□ 세 종류를 나누는 이유는 나중에 run을 찾는 질문이 세 가지로 갈리기 때문임
  ○ " `model_type` 이 `LinearRegression` 인 run만 보여 달라" → 파라미터 칸의 글자를 맞춰 찾음
  ○ " `train_mae` 가 가장 작은 run은 무엇인가" → 메트릭 칸의 숫자를 작은 순으로 나열함. 이 장에서는 `baseline_mean` (1.0)이 `linear` (1.0455)보다 앞에 옴
  ○ "그 run이 쓴 훈련 CSV를 다시 열어 보자" → 아티팩트 칸에 저장된 파일을 내려받음
  ○ 오차를 숫자가 아니라 글자로 적어 두었다면 두 번째 질문처럼 작은 순서로 나열할 수 없음

□ 훈련 결과를 정하는 입력은 코드·데이터·파라미터·환경 넷이며, 이 중 데이터만 자동으로 기록되지 않음
  ○ 코드는 git이 버전으로 남기고, 파라미터와 메트릭은 기록 도구가 남김
  ○ 훈련 데이터는 **파일 경로가 같아도 내용이 바뀜** — 7주차의 정정 재실행이 그런 경우임
  ○ 그래서 훈련 데이터를 정해진 형식의 CSV 텍스트로 만든 뒤 sha256이라는 계산 방식으로 64자의 값을 만들어 두며, 이 값을 **지문**이라 함
  ○ sha256은 내용이 한 글자만 달라도 전혀 다른 64자를 내므로, 두 지문이 같으면 같은 데이터라고 판단함
  ○ 이 지문을 파라미터에 넣어 두면 run마다 어느 데이터를 썼는지 남음

□ 같은 훈련 데이터를 지문과 사본 두 가지로 남기는 것은 쓰임이 다르기 때문임
  ○ 지문(64자)은 짧아서 두 run의 지문을 나란히 놓고 "같은 데이터였나"를 바로 확인함
  ○ 사본(CSV 파일)은 크지만 열어 볼 수 있어서 "그 데이터를 다시 보자"에 쓰임

□ 첫 번째 코드를 실행함. 7주차 확정 집계에서 훈련 데이터를 만들고 지문을 찍음

```
cd practice/chapter9
python3 code/9-2-training-pairs.py
```

```
# 출처: practice/chapter9/results/9-2-training-pairs.log
[1] 원천 – 7주차 확정 집계 3일 × 지역 3개
    경로 practice/chapter7/data/input/ch6_daily/
    일자          마포구(11440)  관악구(11620)  강남구(11680)
    2026-07-01           5           5           8
    2026-07-02           5           5           6
    2026-07-03           6           5           9

[2] 훈련 쌍 – (전일 건수 → 당일 건수) 6개
    지역              전일 → 당일      전일 건수  당일 건수
    마포구(11440)  2026-07-01 → 2026-07-02         5         5
    마포구(11440)  2026-07-02 → 2026-07-03         5         6
    관악구(11620)  2026-07-01 → 2026-07-02         5         5
    관악구(11620)  2026-07-02 → 2026-07-03         5         5
    강남구(11680)  2026-07-02 → 2026-07-03         6         9
    강남구(11680)  2026-07-01 → 2026-07-02         8         6
```

□ [1]·[2]에서 읽을 것

○ **훈련 데이터는 이 장에서 새로 만든 것이 아님** — 6주차 배치가 확정한 지역별 일계를 7주차가 피처로 받았고, 이 장은 그 3일치에서 (전일 → 당일) 쌍 6개를 만들었음. 지역마다 이틀분 전이가 나오므로 3지역 × 2 = 6쌍임

```
# 출처: practice/chapter9/results/9-2-training-pairs.log ([3])
[3] 정규화 CSV와 지문
    첫 줄        lawd_cd,x_prev_count,y_count
    행 수        6행 + 머리글 1행
    sha256       96f6b6b3ce9dcdd1da62e0284e0ceea1ec2afec24c522f6df3221ad0ab31f49e
    9-1 기록값   96f6b6b3ce9dcdd1da62e0284e0ceea1ec2afec24c522f6df3221ad0ab31f49e
    일치 여부    예 – 같은 데이터다
```

□ [3]에서 읽을 것

○ **지문은 CSV를 적는 규칙까지 같아야 같아짐** — 열 이름·열 순서·행 순서·줄바꿈을 정해 둔 CSV 텍스트 전체에 sha256을 계산함. 같은 6쌍이라도 행 순서를 바꿔 적으면 다른 지문이 나오므로, 지문을 쓰려면 적는 규칙부터 고정해야 함

○ **과거 MLflow 실행이 남긴 지문과 같음** — 같은 데이터를 서로 다른 코드로 두 번 계산했는데 64자가 전부 일치함(`sha256`과 `9-1 기록값` 두 줄). 이 일치가 "9-1이 기록한 그 데이터가 지금 저장소에 있는 그 데이터"라는 증거임

```
# 출처: practice/chapter9/results/9-2-training-pairs.log ([4])
[4] 원천 한 칸만 바뀌면 – 마포구 2026-07-03 건수 6 → 7
    바뀐 쌍       (5,6) → (5,7)   나머지 5쌍은 그대로
    sha256        07060a30f8e15e7b8b4f92e502f8acaf2403f0a85118cb5de2b096b364abaa34
    앞에서부터 일치하는 글자 수  0
```

□ [4]에서 읽을 것

○ **한 칸이 바뀌면 지문 전체가 바뀜** — 6을 7로 고쳤을 뿐인데 앞에서부터 일치하는 글자가 0개임. 지문으로는 "얼마나 다른가"를 알 수 없고 같은지 다른지만 알 수 있음. 표 9.2 예측표에 8자나 12자를 적었다면 여기서 틀린 것임

□ 선택 실습을 돌리면 같은 값이 MLflow 기록에도 들어감

```
pip install -r code/requirements.txt
python3 code/9-1-mlflow-tracking.py
```

```
with mlflow.start_run(run_name="linear") as run:
    mlflow.log_params({"model_type": "LinearRegression", "data_sha256_16": fp[:16]})
    mlflow.log_metric("train_mae", mae)
    mlflow.log_artifact(str(csv_path), artifact_path="training_data")
    model_info = mlflow.sklearn.log_model(model, name="model")
```

```
# 출처: practice/chapter9/data/output/ch9_experiment_report.json
{
  "data_fingerprint_sha256": "96f6b6b3ce9dcdd1da62e0284e0ceea1ec2afec24c522f6df3221ad0ab31f49e",
  "mlflow_version": "3.14.0",
  "sklearn_version": "1.9.0",
  "runs": {
    "baseline_mean": { "train_mae": 1.0 },
    "linear": { "train_mae": 1.0455, "coef": { "slope": 0.4091, "intercept": 3.6818 } }
  }
}
```

□ 위 코드와 JSON에서 읽을 것

○ 위 코드에서 `log_params`(파라미터), `log_metric`(메트릭), `log_artifact`(아티팩트)가 서로 다른 함수임. 함수가 나뉘어 있어서 나중에 파라미터는 글자로, 메트릭은 숫자로, 아티팩트는 파일로 따로 찾을 수 있음

○ 지문은 MLflow가 자동으로 넣어 주지 않아 `log_params`에 `data_sha256_16`이라는 이름으로 직접 적었음(지문 앞 16자). 자동으로 남는 것과 사람이 코드로 적어야 남는 것의 차이가 이 줄임

4. 훈련도 함수다 — 같은 입력이면 같은 모델

□ 6주차에서 배치는 같은 입력을 넣으면 산출물 해시까지 같았음. 훈련에도 같은 성질이 있어야 함

- ○ 모델을 만드는 데 들어간 입력은 코드·데이터·파라미터·환경(라이브러리 버전 등) 넷이고, 넷이 전부 같으면 같은 모델이 나와야 함
- ○ 예: 이 장에서 같은 6쌍으로 `linear`를 두 번 훈련하면 두 번 모두 오차 1.0455가 나옴(아래 출력 [5])
- ○ 입력이 하나 더 있을 수 있음 — 훈련 알고리즘이 난수(무작위 숫자)를 쓰면 난수의 시작값(seed)도 같게 고정해야 같은 모델이 나옴. 코드에 눈에 띄게 적히지 않아 놓치기 쉬움
- ○ 이 장의 후보 둘은 난수를 쓰지 않으므로 seed 없이도 같은 값이 나옴

□ 훈련 때 기록을 남기지 않으면 나중에 같은 모델을 다시 만들 수 없음

- ○ 석 달 전 모델의 성능이 떨어져 "그때 모델을 다시 만들어 비교하자"고 할 때, 데이터·파라미터·코드 중 하나라도 확실하지 않으면 다시 만든 모델은 그때 모델과 다른 모델임
- ○ 그러면 "그때와 지금이 어떻게 다른가"를 비교할 수 없음

□ 실험 중에 기록을 자동으로 남겨야 하는 이유는 실험 속도임

- ○ 하루에 파라미터를 다섯 번 바꾸고 데이터를 두 번 바꾸는 일이 흔함
- ○ 그 속도에서 "나중에 정리하자"고 하면 어떤 설정이었는지 기억하지 못하므로, 기록은 실험이 끝난 뒤에 쓰는 문서가 아니라 훈련을 돌리는 코드가 그 자리에서 남기는 기록이어야 함

□ 두 번째 코드를 실행함. 후보 둘을 훈련하고 같은 설정으로 한 번 더 훈련함

```
python3 code/9-3-train-and-rerun.py
```

```
# 출처: practice/chapter9/results/9-3-train-and-rerun.log
[2] 후보 1 baseline_mean — 입력과 무관하게 y 평균을 내놓는다
    예측값       6.0  (= y 합 36 ÷ 6)
    잔차 절댓값  [1.0, 0.0, 1.0, 1.0, 3.0, 0.0]
    train_mae    1.0000

[3] 후보 2 linear — 전일 건수의 1차 함수
    slope        0.4091  (= 9/22)
    intercept    3.6818  (= 81/22)
    train_mae    1.0455
    → 복잡한 쪽이 기준선보다 나쁘다: 1.0455 > 1.0000
```

□ [2]·[3]에서 읽을 것

- ○ **전일 건수를 쓰는 모델의 오차가 더 컸음** — `linear`(전일 건수로 당일 건수를 예측, 오차 1.0455)가 `baseline_mean`(입력과 무관하게 평균 6.0만 내놓음, 오차 1.0000)보다 큼. 훈련 쌍이 6개뿐이어서 전일 건수와 당일 건수의 관계를 제대로 배우지 못한 결과이며, 5절의 게이트 판정에서 이 두 값을 비교함

○ **계수가 분수로 정확히 나옴** — 기울기 9/22, 절편 81/22는 오차 제곱의 합을 가장 작게 하는 직선(최소제곱법)을 손으로 푼 값임. 아래 [4]에서 라이브러리가 낸 값과 소수점 넷째 자리까지 같음을 확인함

```
# 출처: practice/chapter9/results/9-3-train-and-rerun.log ([4])
[4] 9-1(scikit-learn)이 남긴 값과 대조
    baseline_mean train_mae    손계산 1.0      기록 1.0      일치 예
    linear train_mae           손계산 1.0455   기록 1.0455   일치 예
    linear slope               손계산 0.4091   기록 0.4091   일치 예
    linear intercept           손계산 3.6818   기록 3.6818   일치 예
```

□ [4]에서 읽을 것

○ 손으로 계산한 값 넷이 scikit-learn이 남긴 값과 모두 일치함. 모델이 무엇을 계산했는지 직접 검산할 수 있다는 뜻임

```
# 출처: practice/chapter9/results/9-3-train-and-rerun.log ([5])
[5] 같은 입력으로 다시 훈련(재현성)
    지문         96f6b6b3ce9dcdd1… → 96f6b6b3ce9dcdd1…
    train_mae    1.0455 → 1.0455   동일 True
    계수         (0.4091, 3.6818) → (0.4091, 3.6818)   동일 True
```

□ [5]에서 읽을 것

○ **다시 돌려도 같은 값이 나옴** — 지문도 오차도 계수도 동일함. 6주차의 "재실행 산출물 해시 동일"이 여기서 "재훈련 메트릭·계수 동일"이 된 것이며 같은 원리임

```
# 출처: practice/chapter9/results/9-3-train-and-rerun.log ([6])
[6] 입력이 한 건 바뀐 채로 같은 코드를 돌리면 – 마포구 마지막 날 건수 +1
    지문         96f6b6b3ce9dcdd1… → 07060a30f8e15e7b…   같음 False
    baseline_mean train_mae  1.0000 → 1.2222
    linear train_mae         1.0455 → 1.2576
    linear slope             0.4091 → 0.3182
```

□ [6]에서 읽을 것

○ **입력이 한 건 바뀌면 지문과 메트릭이 함께 바뀜** — 지문이 달라졌고 오차는 1.0000에서 1.2222로, 기울기는 0.4091에서 0.3182로 바뀜. 6건 중 1건이 바뀌었을 뿐인데 학습된 함수가 달라짐

□ 이 확인은 "지난달과 같은 설정으로 돌렸는데 오차가 다르다"는 신고가 들어왔을 때 원인을 찾는 데 씀

○ 기록이 있으면 짐작하지 않고 아래 순서로 두 run을 비교함. 비교 비용이 싼 것부터 함

– 1단계, 두 run의 파라미터를 항목별로 비교함. 다르면 거기서 원인이 확인됨. "같은 설정으로 돌렸다"는 사람의 기억이고, 파라미터 기록은 실제로 넣은 값임

– 2단계, 파라미터가 전부 같은데 데이터 지문이 다르면 데이터가 바뀐 것임. 파일 경로가 같아도 내용은 다를 수 있음(출력 [6]에서 건수 한 건이 바뀌자 오차가 1.0455에서 1.2576으로 바뀜)
– 3단계, 1·2단계가 같으면 라이브러리 버전을 비교함. 수치 계산 라이브러리는 버전이 달라지면 값이 조금 달라질 수 있음
– 4단계, 셋 다 같은데 결과가 다르면 seed를 고정하지 않은 난수를 찾음

□ 1·2단계의 기록은 **훈련할 때** 남겨 두어야 하고, 오차가 다르다는 신고가 들어온 뒤에는 만들 수 없음
○ 정보공개 청구나 감사에서 "이 모델의 판단 근거를 소명하라"는 요구가 오면, 모델 파일만 가진 기관은 어떤 데이터와 설정으로 만들었는지 답하지 못함
○ 그러므로 재현성은 연구자의 취향이 아니라, 문제가 생겼을 때 누가 무엇을 근거로 결정했는지 밝히기 위한 조건임

5. 승격 게이트는 통과 규칙이 아니라 교체 규칙이다

□ 이 절은 세 가지를 차례로 다룸 — 모델 레지스트리, 별칭, 승격 게이트
○ 세 가지는 "운영 중인 모델이 무엇인가"라는 질문 하나에서 차례로 나옴

5.1 모델 레지스트리 — 후보에 이름과 번호를 붙인다

□ 4절까지 훈련하면 run이 둘 생겼음 — `baseline_mean`(train_mae 1.0)과 `linear`(train_mae 1.0455)
○ 실험을 계속하면 run은 수십, 수백 개로 늘어남
○ 서빙 시스템(운영 중 예측 요청을 받는 프로그램)에 "run 목록에서 알아서 좋은 것을 골라 쓰라"고 하면, 어느 run을 쓰는지 아무도 모르게 됨

□ 그래서 배포 후보만 따로 등록하는 저장소를 두며, 이것이 **모델 레지스트리(registry)** 임
○ 등록할 때 모델에 이름 하나를 붙임 — 이 장의 이름은 `complaint_daily_forecaster`임
○ 같은 이름으로 등록할 때마다 번호가 1씩 늘어난 **버전**이 생김 — `baseline_mean`은 v1, `linear`는 v2가 됨
○ 각 버전에는 그 버전을 만든 run의 번호가 적혀 있어서, 3절에서 남긴 파라미터·메트릭·데이터 지문을 버전에서 바로 찾아감

5.2 별칭 — 서빙 코드가 버전 번호를 모르게 한다

□ 버전이 여러 개이면 "지금 운영 중인 것은 v1인가 v3인가"를 한 곳에서 답해야 함
○ 서빙 코드에 `v1`을 적어 두면, 모델을 바꿀 때마다 서빙 코드를 고쳐서 다시 배포해야 함
○ 그래서 버전에 **별칭(alias)** 을 붙임 — 별칭은 버전 하나에 붙이는 이름표이며, 같은 별칭을 다른 버전으로 옮길 수 있음

□ 이 장은 운영 중인 버전에 `champion`이라는 별칭을 붙임
○ 서빙 코드(`9-1-mlflow-tracking.py` 220행)는 버전 번호 없이 이름과 별칭만 적어 모델을 불러옴

```
loaded = mlflow.pyfunc.load_model(f"models:/{MODEL_NAME}@champion")
```

□ `champion`을 v1에서 v3으로 옮기면 위 한 줄은 그대로인데 불러오는 모델이 v3으로 바뀜
  ○ 이 장의 로그에서는 같은 입력(강남구 전일 건수 9.0)에 v1이 6.00, v3이 7.50을 냄(아래 출력 [6])
  ○ 그래서 별칭을 옮기는 일은 코드 수정 없이 일어나는 **배포**이고, 옮긴 이력을 남겨야 함
  ○ 7주차에서 서빙이 피처를 정의 이름으로 조회했듯, 여기서는 모델을 별칭으로 조회함

5.3 승격 게이트 — 별칭을 옮겨도 되는지 정하는 규칙

□ 새 후보가 등록될 때마다 `champion`을 옮길지 사람이 그때그때 정하면 기준이 사람마다 달라짐
  ○ 그래서 옮기는 조건을 미리 적어 두며, 이 조건이 **승격 게이트**임
  ○ 게이트의 조건은 두 종류임
□ **절대 조건**은 후보 하나만 보고 정함
  ○ 예: "train_mae가 1.05 이하일 것"
  ○ 질문은 "이 모델을 운영에 써도 되는 수준인가"임
□ **상대 조건**은 지금 `champion`과 비교해서 정함
  ○ 예: "train_mae가 현재 champion(1.0)보다 작을 것"
  ○ 질문은 "지금 모델을 이 모델로 바꿀 이유가 있는가"임
□ 두 질문은 서로 다르므로 하나만 두면 빠지는 경우가 생김
  ○ 절대 조건만 두면 지금보다 나쁜 모델이 조건만 맞춰 들어옴
  ○ 상대 조건만 두면 업무 기준에 못 미치는 모델끼리 바뀌기만 함
  ○ 아래 실행에서 두 경우를 숫자로 확인함
□ 세 번째 코드를 실행함. 같은 후보에 규칙 셋을 각각 적용해 판정이 갈리는지 봄

```
python3 code/9-4-gate-matrix.py
```

```
# 출처: practice/chapter9/results/9-4-gate-matrix.log
[1] 후보 3개 — 같은 훈련 쌍 6개, 지문 96f6b6b3ce9dcdd1…
    후보              train_mae  입력으로 요구하는 것  설명
    baseline_mean       1.0000  없음            훈련 y 전체 평균 — 입력을 보지 않는다
    linear              1.0455  전일 건수         전일 건수의 1차 함수
    district_mean       0.6667  지역 코드         지역별 y 평균 — 지역마다 다른 값을 낸다
```

□ [1]에서 읽을 것
  ○ 후보는 셋이고 오차는 `district_mean`(0.6667)이 가장 작고 `baseline_mean`(1.0000), `linear`(1.0455) 순임

○ `입력으로 요구하는 것` 열에 따르면 `district_mean`은 예측할 때 지역 코드를 넣어야 함. 이 차이는 아래 [7]에서 문제가 됨

```
# 출처: practice/chapter9/results/9-4-gate-matrix.log ([3])
[3] 같은 후보를 세 규칙으로 판정 — champion은 baseline_mean(1.0000)
    규칙                          linear(1.0455)         district_mean(0.6667)
    A 절대만 (mae <= 1.05)          승격                     승격
    B 상대만 (champion보다 개선)       반려(개선 없음)             승격
    C 둘 다                       반려(개선 없음)             승격
```

□ [3]에서 읽을 것

○ **규칙 셋이 같은 후보에 다른 판정을 냄** — `linear`는 규칙 A(절대만)에서 승격, 규칙 B·C에서 반려임. `district_mean`은 세 규칙 모두 승격임

○ **둘을 함께 두어야 판정이 업무와 맞음** — 규칙 C에서 `linear`는 반려되고 `district_mean`만 승격됨. 절대 조건이 "쓸 만한가"를 묻고 상대 조건이 "바꿀 이유가 있는가"를 물음

```
# 출처: practice/chapter9/results/9-4-gate-matrix.log ([4])
[4] 규칙 A(절대만)를 쓰면 무엇이 일어나는가
    linear 1.0455 <= 1.05 이므로 통과 → champion이 1.0000에서 1.0455로 교체됨
    운영 모델의 오차가 커졌는데도 규칙은 정상 작동했다고 기록된다
```

□ [4]에서 읽을 것

○ **절대 조건만 두면 더 나쁜 모델이 운영에 들어감** — `linear`는 1.0455로 임계 1.05를 넘지 않아 통과하고, champion이 1.0000인 모델에서 1.0455인 모델로 교체됨. 오차가 커졌는데 로그에는 "게이트 통과"만 남음. 예측표 셋째 줄에 "아니오"를 적었다면 여기서 어긋남

```
# 출처: practice/chapter9/results/9-4-gate-matrix.log ([5])
[5] 임계값을 업무 기준으로 조이면 — mae <= 0.8
    baseline_mean       1.0000  미달
    linear              1.0455  미달
    district_mean       0.6667  통과
    등록 순서가 반대여서 linear가 먼저 champion이 되었다면, 상대 조건만으로는
    baseline_mean(1.0000 < 1.0455) 판정이 '승격' — 둘 다 업무 기준 미달인데 교체가 일어난다
```

□ [5]에서 읽을 것

○ **상대 조건만 두면 못 쓸 모델끼리 교체가 계속됨** — 업무가 요구하는 오차가 0.8이라면 `baseline_mean`도 `linear`도 미달임. 그런데 둘 중 나은 쪽으로 바꾸는 규칙은 계속 작동해, 운영에 쓸 수 없는 모델이 바뀌기만 함

```
# 출처: practice/chapter9/results/9-4-gate-matrix.log ([6])
[6] 별칭 이동과 롤백 — 규칙 C로 district_mean을 v3로 등록한 뒤
    champion=v1  강남구 전일 건수 9.0 → 예측 6.00
    champion=v3  강남구 전일 건수 9.0 → 예측 7.50
    별칭 이력    v1 → v3 → (롤백) v1     소비자 코드는 한 줄도 바뀌지 않는다
    반려된 v2를 지우지 않았기 때문에 되돌아갈 수 있는 버전이 v1·v2 둘이다
```

☐ [6]에서 읽을 것

○ **별칭을 옮기면 같은 입력의 답이 바뀜** — 강남구 전일 건수 9.0에 champion v1은 6.00을, v3은 7.50을 냄. 소비자 코드는 그대로인데 결과가 달라지므로, 별칭 이동은 배포 행위이며 이력이 남아야 함

○ **반려된 버전을 지우지 않아야 롤백할 수 있음** — 롤백은 `champion`을 이전 버전으로 되돌리는 일임. 이 출력에서는 v3으로 옮겼다가 v1로 되돌렸고, 지우지 않은 v1·v2가 되돌아갈 수 있는 버전으로 남아 있음. 버전을 지우면 되돌아갈 곳이 없어짐

```
# 출처: practice/chapter9/results/9-4-gate-matrix.log ([7])
[7] 지표가 좋아졌다고 끝나지 않는다 — district_mean의 조건
    훈련에 있는 지역  11440, 11620, 11680
    처음 보는 지역 11110을 넣으면  예측 None
```

- **[7]에서 읽을 것 — 오차가 작다고 운영에 맞는 모델은 아니다**
- `district_mean`은 오차가 0.6667로 가장 작지만 예측에 지역 코드를 요구하고, 훈련에 없던 지역 11110을 넣으면 값을 내지 못한다.
- 게다가 지역별 표본이 2개뿐이다. 게이트 지표 하나로 판정하면 이런 조건은 드러나지 않으므로, 승격 규칙에는 지표 외의 요건도 함께 적는다.

☐ 9-1을 실행한 환경에서는 같은 판정이 레지스트리에 기록으로 남음

```
# 출처: practice/chapter9/data/output/ch9_promotion_record.json
{
  "model_name": "complaint_daily_forecaster",
  "champion_version": 1,
  "gate_rule": "train_mae <= 1.05 그리고 기존 champion 대비 개선",
  "decisions": [
    { "version": 1, "train_mae": 1.0,    "metric_gate_pass": true, "promoted": true,
      "action": "champion 별칭 부여(승격)" },
    { "version": 2, "train_mae": 1.0455, "metric_gate_pass": true, "promoted": false,
      "action": "승격 반려 – 절대 게이트 통과했으나 기존 champion 대비 개선 없음" }
  ]
}
```

□ 위 JSON에서 읽을 것

○ v2(`linear`)는 `metric_gate_pass`가 `true`(절대 조건 통과: 1.0455 ≤ 1.05)인데 `promoted`는 `false`(교체 안 함: champion 1.0보다 개선되지 않음)임

○ 두 값을 한 칸에 합쳐 "통과 여부"로만 적었다면, 나중에 "통과했는데 왜 안 바꿨는가"라고 물을 때 기록에서 답을 찾을 수 없음

6. 승격은 기술 행위가 아니라 승인 행위다

□ 앞의 세 절에서 남긴 기록은 "어떤 데이터와 설정으로 만들었고 오차가 얼마인가"에 답함

○ 공공 기관의 감사(기록이 규정에 맞는지 외부에서 점검하는 일)는 여기에 네 가지를 더 물음

○ **누가** 만들었는가, **무슨 목적**으로 만들었는가, **어느 기간의 어떤 출처** 데이터를 썼는가, **어떤 제한** 아래 쓰는가

□ 이 네 가지는 오차와 무관하지만 책임을 가리는 데 쓰이고, 나중에 쓰려 하면 쓸 수 없음

○ 담당자가 다른 부서로 옮기면 목적과 데이터 출처를 아는 사람이 없어짐

○ 그래서 훈련할 때 run에 태그(tag, 이름표 형태로 적는 짧은 글자 항목)로 함께 적어 둠

○ 이 장의 실습이 적는 태그는 소유자·목적·데이터 출처·데이터 기간 네 개임

□ 승격을 결정한 기록도 남김

○ 공공 기관에서 승격은 "이 모델을 운영에 써도 된다"는 승인이므로, 누가 어떤 규칙으로 어떤 근거를 보고 통과시켰는지를 기록으로 남겨야 함

○ 이 기록은 사람이 나중에 쓰는 보고서가 아니라, 판정하는 코드가 판정하는 순간에 출력한 것이어야 함. 나중에 쓰면 기억으로 채우게 되기 때문임

□ 검수 기록에는 "같은 설정으로 다시 돌리면 같은 결과가 나오는가"라는 항목도 들어감

○ 게이트는 오차 수치를 보고 통과 여부를 정하고, 재현성 항목은 그 수치를 믿어도 되는지를 확인함

○ **다시 돌렸을 때 같은 수치가 나오지 않는 run은 오차가 좋아도 승격하지 않음** — 그 오차가 코드와 데이터 때문에 나온 것인지 우연히 나온 것인지 가릴 수 없기 때문임

□ 네 번째 코드를 실행함. 승격 검수 기록이 감사 질문에 답할 수 있는지 점검함

```
python3 code/9-5-review-record-check.py
```

```
# 출처: practice/chapter9/results/9-5-review-record-check.log
[2] 감사 질문별 점검
    통과  항목 model_name          어느 모델인가
    통과  항목 champion_version    지금 운영 중인 것은 몇 번인가
    통과  항목 gate_rule           어떤 규칙으로 통과시켰는가
    통과  항목 decisions           각 버전을 어떻게 판정했는가
    통과  항목 reproducibility_check 그 수치가 다시 나오는가
    통과  항목 data_fingerprint_sha256 어느 데이터로 훈련했는가
    통과  항목 reviewer            누가 확인했는가
    통과  항목 notes               어떤 제한이 붙는가
    통과  지문 재계산 대조              기록된 지문이 원천에서 다시 나오는가
          기록 96f6b6b3ce9dcdd1… / 재계산 96f6b6b3ce9dcdd1…
    통과  champion 일치            champion 버전이 승격 판정과 맞는가
          champion v1 / 승격 판정 [1]
    통과  반려 이력 보존               반려된 버전이 기록에 남아 있는가
          반려 [2] – 롤백 대상이자 재시도 방지 근거
    통과  재현성 통과                  같은 설정으로 같은 결과가 나왔는가
          mae_equal=True coef_equal=True
    미흡  검수자 실명                  확인한 사람이 특정되는가
          reviewer='검수 담당(가상 – 시뮬레이션 표기)' – 실운영이면 이 칸이 비면 안 된다
    → 12/13 통과
```

□ [2]에서 읽을 것

○ **점검 항목 하나가 감사 질문 하나와 짝을 이룸** — 13개 항목 각각이 "어느 모델인가", "누가 확인했는가" 같은 질문 하나에 대응함. 감사에서 나올 질문을 먼저 목록으로 적고, 그 질문마다 기록 파일에 칸을 하나씩 만들면 이 점검표가 됨

○ **지문을 다시 계산해 맞춰 보면 기록이 사실인지 확인됨** — 기록에 적힌 64자를 원천 자료에서 다시 계산해 비교함. 기록만 읽는 점검은 "값이 적혀 있다"까지만 확인하고, 다시 계산하는 점검은 "그 값이 맞다"까지 확인함

○ **12/13에서 걸린 한 칸이 사람 칸임** — 나머지는 자동으로 채워지는데 검수자만 비었음. 교육용이라 가상 표기로 두었지만, 행정 처분이나 자원 배분에 쓰이는 모델이라면 이 칸이 비면 안 됨

```
# 출처: practice/chapter9/results/9-5-review-record-check.log ([3])
[3] 기록이 부실할 때 — reviewer를 빼고 지문을 다른 값으로 바꾼 사본
    미흡  항목 reviewer            누가 확인했는가
    미흡  지문 재계산 대조              기록된 지문이 원천에서 다시 나오는가
          기록 0000000000000000… / 재계산 96f6b6b3ce9dcdd1…
    미흡  검수자 실명                 확인한 사람이 특정되는가
    → 10/13 통과
```

□ [3]에서 읽을 것

○ **점검이 실제로 잡아냄** — 검수자를 빼고 지문을 다른 값으로 바꾼 사본은 10/13으로 내려감. 통과 수가 줄어드는 것을 확인해야 그 점검이 형식이 아님을 알 수 있음

□ 이 점검표를 실제 기관에 적용하려면 세 가지를 정해야 함

○ 게이트를 통과하면 승격을 자동으로 할 것인가, 사람이 확인한 뒤에 할 것인가

○ 게이트에 못 미치는데 승격해야 하는 사정이 생기면 누가 예외를 승인하는가

○ `champion` 별칭을 옮길 권한을 누가 갖는가

□ 세 번째가 특히 중요함

○ **모델을 훈련한 사람이 별칭도 옮기면 게이트를 통과시키는 사람과 통과하는 사람이 같아져, 외부 확인 없이 자기가 자기를 승인하게 됨**

7. 핵심 요약

□ **1. run 하나에는 파라미터·메트릭·아티팩트를 나누어 남기고, 훈련 데이터는 지문을 직접 계산해 적어야 한다.** (3절)

○ 입력 설정과 결과 수치와 결과 파일은 찾는 방식이 달라 나누어 남김

○ 코드는 git이, 파라미터는 도구가 자동으로 남기지만 훈련 데이터는 경로가 같아도 내용이 바뀌므로 sha256 지문을 파라미터로 실어 보냄

○ 원천 한 칸을 6에서 7로 고쳤더니 지문 앞 16자 중 일치하는 글자가 0개였음

□ **2. 같은 입력이면 같은 모델이 나오고, 안 나오면 무엇이 바뀌었는지 기록으로 가려낼 수 있다.** (4절)

○ 선형회귀를 다시 훈련해도 오차 1.0455와 계수 (0.4091, 3.6818)가 그대로였음

○ 입력을 한 건만 바꾸자 지문이 달라지고 오차가 1.0000에서 1.2222로 바뀌었음

○ 파라미터 → 지문 → 환경 → 난수 순으로 대조하되, 앞의 둘은 훈련 시점에 남겨야만 존재함

□ **3. 절대 게이트와 상대 조건은 다른 질문에 답하므로 둘 다 필요하다.** (5절)

○ 절대 게이트만 두면 1.0455가 임계 1.05를 통과해 1.0000인 현행 모델이 교체됨

○ 상대 조건만 두면 업무 기준 0.8에 둘 다 미달인 후보끼리 교체가 계속됨

○ 둘을 함께 둔 규칙 C에서만 linear가 반려되고 district_mean이 승격됐음

□ **4. 별칭 이동이 배포이고, 반려 버전을 남기는 것이 롤백 준비다.** (5절)

○ 같은 입력 9.0에 champion v1은 6.00을, v3은 7.50을 냈고 소비자 코드는 바뀌지 않았음

- ○ 반려된 v2를 지우지 않았기에 되돌아갈 수 있는 버전이 둘로 남았음

□ **5. 승격은 승인 행위이고, 검수 기록은 감사 질문 목록을 파일로 옮긴 것이다.** (6절)

- ○ 13개 점검 중 12개가 자동으로 통과했고 걸린 한 칸이 검수자였음
- ○ 지문을 원천에서 다시 계산해 맞춰야 기록의 진위까지 확인됨
- ○ 훈련하는 사람과 champion 별칭을 옮기는 사람이 같으면 게이트는 자기 검사에 그침

8. 과제

□ 실습을 순서대로 실행해 만든 산출물을 LMS에 제출한다

1. 표 9.2 예측표 — 실행 결과와 대조해 채운 최종본. 텍스트든 이미지든 형식은 무방하다.
2. `practice/chapter9/data/output/ch9_training_runs.json`
3. `practice/chapter9/data/output/ch9_gate_matrix.json`
4. `practice/chapter9/data/output/ch9_record_audit.json`
5. `practice/chapter9/data/output/ch9_experiment_report.json`

□ 5번은 선택 실습(`9-1-mlflow-tracking.py`)의 산출물이다

- ○ MLflow 설치가 막혀 실행하지 못했으면 저장소에 포함된 파일을 그대로 내고, 어디서 막혔는지 한 줄로 적는다

□ 이어서 네 가지를 각각 세 문장 이내로 쓴다

1. `ch9_training_runs.json` 의 `data_fingerprint_sha256` 과 `input_changed_one_cell.data_fingerprint_sha256` 을 나란히 옮겨 적고, 훈련 데이터가 바뀐 것을 지문 없이 알아내려면 무엇이 필요한지 쓴다.
2. `ch9_gate_matrix.json` 의 `gate_matrix` 에서 규칙 A와 규칙 C의 `linear` 판정을 옮겨 적고, 두 판정이 갈린 이유를 쓴다.
3. `ch9_gate_matrix.json` 의 `alias_move` 에서 `forecast_champion_v1` 과 `forecast_champion_v3` 를 옮겨 적고, 별칭을 옮길 때 소비자에게 무엇을 알려야 하는지 쓴다.
4. `ch9_record_audit.json` 에서 `pass` 가 `false` 인 항목을 찾아 이름을 적고, 그 칸을 채우려면 누가 무엇을 해야 하는지 쓴다.

□ 값을 지어내지 않는다

□ 실행이 안 되면 어디서 막혔는지 그대로 쓴다

□ 수업일 포함 7일 이내에 제출한다

9. 더 알아보기

□ 실습을 더 해 보려면 `practice/chapter9/` 에서 다음을 시도함

- ○ `9-4-gate-matrix.py`의 `GATE_BUSINESS` 값을 바꿔, 임계를 몇으로 두면 어떤 후보도 통과하지 못하는지 찾음
- ○ `9-2-training-pairs.py`의 정규화 규칙(열 순서·정렬 기준)을 바꿔, 내용이 같은데도 지문이 달라지는 것을 확인함
- ○ `9-5-review-record-check.py`의 `REQUIRED` 표에 항목을 더해, 자기 기관이라면 어떤 질문을 더 물을지 점검 항목으로 옮김
- ○ MLflow를 설치했다면 `mlflow ui --backend-store-uri sqlite:///data/output/mlflow.db`로 run 목록과 아티팩트를 화면에서 봄

□ 뒤 주차에서 이어지는 내용

- ○ 승격된 모델을 실제 서비스로 내보내고 되돌리는 절차는 10주차, 배포 후 성능이 경고 없이 나빠지는 것을 잡는 일은 11주차에서 실행함
- ○ 피처 수준의 권한·이력 관리는 12주차, 이 장의 기록을 이용자에게 설명하는 문제는 14주차에서 이어짐

□ 원문을 직접 보고 싶다면 다음 두 편이 출발점임

- ○ MLflow Documentation,「Model Registry」— 버전·별칭·단계의 정의. https://mlflow.org/docs/latest/ml/model-registry/
- ○ Mitchell 외,「Model Cards for Model Reporting」— 모델을 외부에 설명하는 공개 양식. https://arxiv.org/abs/1810.03993